“You won’t all become physicists, and not every programmer uses physics every day. But whenever software controls movement, reads a sensor, processes sound or creates a virtual world, physical quantities matter. Physics also teaches you to check whether an answer makes sense—not just whether your code runs.”

Real-world examples they can relate to

Application	Where physics enters	Connection to your lessons
Video games	Characters accelerate, jump, fall and collide.	Position, velocity and acceleration determine how movement feels.
Robotics and automated machinery	A robot must move to a position, change speed and stop.	1D kinematics is a starting point for controlling one wheel, joint or sliding mechanism.
Fitness and navigation apps	Software turns position and time measurements into speed and distance.	Units, displacement, average speed and measurement uncertainty.
IoT and sensor systems	Sensors report temperature, distance, voltage and acceleration.	Values need units, conversion and checks before software can use them reliably.
User-interface animation	Menus, scrolling and transitions speed up and slow down.	Position changing over time; velocity and acceleration help describe smooth motion.
Computer vision	Software tracks an object across video frames.	Position changes and frame timing estimate velocity. Converting pixels to metres requires calibration.
Audio and music software	Microphones record waves; software filters, analyses and reproduces them.	Later physics topics: frequency, amplitude, waves and sampling.
Computer hardware	Processors consume electrical power and produce heat.	Later topics: electricity, energy and heat explain physical limits on computing.

Three examples you can use immediately

1. “Why does this game character move faster on a better computer?”

Suppose a game moves a character **5 pixels every frame**:

- At 30 frames per second: 150 pixels per second.

- At 120 frames per second: 600 pixels per second.

The programmer has confused movement *per frame* with movement *per second*.

For constant velocity, the fix uses today's physics:

position = position + velocity * elapsed_time

Here, velocity is in pixels per second and elapsed time is in seconds. Ask: **"Which units cancel, and which remain?"**

2. "The code works—but the robot goes to the wrong place."

A distance sensor reports `250`, meaning **250 millimetres**. Another function interprets it as **250 metres**.

There may be no Python error at all, but the interpreted distance is **1,000 times too large**.

Ask: **"Does a number tell us enough without its unit?"**

This makes unit conversion feel like a software correctness issue, not just an arithmetic exercise.

3. "How does a robot know when to start slowing down?"

In a simplified constant-acceleration model, a small cart moves at 2 m/s2\text{ m/s} and slows with acceleration −1 m/s2-1\text{ m/s}^2.

It takes **2 seconds** to stop and travels **2 metres** while doing so.

Ask students to predict its velocity-time graph, then calculate the triangular area underneath it. Explain that real control systems must also account for sensing delays, changing conditions and safety margins.

A useful closing line

> "Programming tells the computer what to calculate. Physics helps you decide what to calculate—and whether the result is believable."